SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

ZAVRŠNI RAD br. 2045

RAZVOJ VST DODATKA ZA SINTEZU ZVUKA TAMBURE

Josip Skledar

Zagreb, lipanj 2025.

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

ZAVRŠNI RAD br. 2045

RAZVOJ VST DODATKA ZA SINTEZU ZVUKA TAMBURE

Josip Skledar

Zagreb, lipanj 2025.

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

Zagreb, 3. ožujka 2025.

ZAVRŠNI ZADATAK br. 2045

Pristupnik:	**Josip Skledar (0036552638)**
Studij:	Elektrotehnika i informacijska tehnologija i Računarstvo
Modul:	Računarstvo
Mentor:	prof. dr. sc. Kristian Jambrošić

Zadatak: **Razvoj VST dodatka za sintezu zvuka tambure**

Opis zadatka:

Razvoj kvalitetnih algoritama za sintezu zvuka glazbenih instrumenata predstavlja važan aspekt moderne audio produkcije. U MIDI okruženju, vjerodostojna reprodukcija karakterističnih zvukova tradicionalnih instrumenata često predstavlja izazov, posebice za instrumente koji nisu široko zastupljeni u standardnim bazama glazbala. Tambura, kao središnji instrument hrvatske tradicijske glazbe, često ne dobiva adekvatnu reprezentaciju u digitalnom audio okruženju. Stvaranje specijaliziranog VST dodatka za sintezu zvuka tambure omogućilo bi glazbenicima korištenje autentičnog zvuka ovog glazbala u svojim projektima. Cilj ovog završnog rada je razviti VST dodatak za sintezu zvuka tambure koristeći programsko okruženje JUCE ili ekvivalentno razvojno okruženje koje podržava izradu VST dodataka. Dodatak treba vjerno oponašati akustičke karakteristike različitih vrsta tambura, te omogućiti kontrolu parametara koji definiraju zvuk ovog grupe instrumenata. Implementirajte mogućnost podešavanja vremenske ovojnice sintetiziranog zvuka tambura, te način trzanja žice.

Rok za predaju rada: 23. lipnja 2025.

Sadržaj

Uvod 1

1. Karplus-Strong algoritam 2

1.1. Implementacija pomoću gitarske pedale 2

1.2. Osnovni Karplus-Strong algoritam 3

1.3. Interpolacija uzoraka 4

2. Oblikovanje zvuka 6

2.1. Alikvotni tonovi 6

2.2. Ovisnost izlaza o pobudi 7

2.3. Vrste tambura i štimovi 8

2.4. Načini sviranja 9

2.5. Ubacivanje snimljenog impulsa tambure 9

2.6. Prigušenje povratne veze 11

3. Korištene tehnologije 13

3.1. Razvojna okolina JUCE 13

3.2. Programski jezik C++ 13

3.3. MIDI sučelje 13

3.4. Audacity i Cakewalk 14

3.5. Spek 14

3.6. Sučelje VST 14

3.7. ASIO driveri 14

4. Razvoj 15

4.1. Program ukratko 15

4.2. Klasa TamburaApp 15

4.3. Klasa StringSynthesiser 18

Zaključak 25

Literatura 26

Sažetak 27

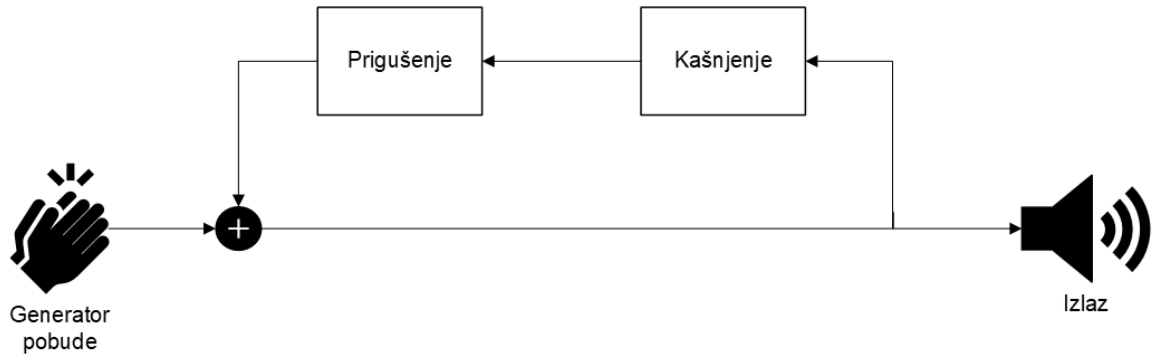
Summary 28

Uvod

1983. godine Karplus i Strong objavljuju članak *Digitalna sinteza tonova trzanih žica i bubnjeva* [1]. Smetalo im je što za sintezu neperiodičnih zvukova treba skupo računalo ili digitalni sintesajzer. Zbog toga, glazbenicima i eksperimentatorima nije bila dostupna sinteza „prirodnih“ instrumenata, već samo periodičnih tonova koji se beskonačno dugo ponavljaju i samim time ne postoje u prirodi i ne zvuče kao klasični instrumenti. Uspjeli su naći elegantan način da s malo memorije i malo procesorske moći (koliko treba za samo par oscilatora sinusnog signala), na računalu proizvedu zvuk koji zvuči kao žica. Zanimljivo da je to tek tada otkriveno, jer se metoda čini jednostavna: pustiti kratak šum kroz vrlo kratak sklop za kašnjenje s polaganim prigušenjem. Ponukan time što danas ima jako malo vjerodostojnih sinteza zvukova tambura, krećem u izradu ovog rada. Dok dirigenti i voditelji KUD-ova rade u programima za pisanje nota, koriste virtualne violine i gitare da dobiju uvid u to kako njihov aranžman zvuči. Karplus-Strong algoritam se dugi niz godina koristio kao baza za vjerodostojne sinteze gitara i logično je njega primijeniti na ovaj problem. Naći ćemo način za pobuđivanje sustava za dobivanje što realnijeg zvuka korištenjem snimaka impulsa pravih tambura. Za implementaciju algoritma koristi se programski jezik C++ i razvojni okvir JUCE. Cilj je razviti aplikaciju koja prima MIDI poruke i proizvodi zvukove tambura: bisernice, brača, bugarije i basa. Treba omogućiti korisniku namještanje parametara poput prigušenja odzvanjanja tona, brzine trzanja i odabira instrumenta.

1. Karplus-Strong algoritam

Način rada je vrlo jednostavan: napravimo petlju i pobudimo je šumom (npr. pljesak). Malo detaljnije, napravimo sustav koji možemo pobuditi impulsom, koji u sebi ima povratnu vezu s kašnjenjem i prigušenjem, prikazan na slici ispod (Slika 1). Kašnjenjem određujemo frekvenciju tona, period generiranog tona jednak je duljini kašnjenja. Prigušenjem, polaganim smanjenjem amplitude signala dobiva se efekt prirodnog iščezavanja žice.



Slika 1 Pojednostavljena shema Karplus-Strong algoritma

Smanjenje se može razlikovati po frekvencijama i to određuje karakteristiku tona, zajedno s pobudom koju dovedemo na ulaz. Nećemo se fokusirati na oblikovanje različito prigušenje po frekvencijama, već samo utjecaj ulazne pobude. Pobuda može biti šum, a u računarstvu može biti slučajan niz bitova. Ako je kao pobuda doveden bijeli šum, na izlazu se dobije jako grubi, metalni ton s puno alikvotnih tonova (vidi 2.1.). Bolje rezultate dobivamo ako na pobudu stavimo nisko propusni filtar s polaganim padom. Zanimljivo je da šum mora sadržavati osnovni ton i njegove prirodne alikvotne tonove kako bi se uopće pojavili na izlazu. Ako pojasnom branom izbacimo samo osnovni ton, na izlazu ćemo dobiti samo alikvotne tonove. Zato je potrebno da šum pobude sadrži sve frekvencije koje instrument koji modeliramo može proizvesti, uključujući alikvote.

1.1. Implementacija pomoću gitarske pedale

Moguće je simulirati Karplus-Strong algoritam samo pomoću gitarske *delay* pedale za kašnjenje. Ona najčešće ima parametre: *time* (vrijeme kašnjenja), *feedback* (povratna veza, 0% za maksimalno prigušenje, 100% za bez prigušenja) i *blend* (miješalo, koliko će biti glasan ulazni signal naspram vraćenog). Vrijeme kašnjenja računamo ovisno o frekvenciji.

Ako želimo proizvesti ton frekvencije 200 Hz, vrijeme kašnjenja postavit ćemo na 1/200 = 5 milisekundi. Feedback diktira koliko brzo ton „umire“. Ovaj parametar možemo interpretirati kao broj puta koliko se ton mora ponoviti prije nego što utihne. Iz toga trebamo zaključiti da što je vrijeme kašnjenja kraće, to feedback mora biti veći. *Blend* možemo ostaviti na 50%, da čujemo i pobudu i stvoreni ton. Ako na ulazu generiramo šum generiranjem bilo kakve pobude, npr. diranjem kontakta kabla golom rukom, zaista se čuje ton na izlazu.

1.2. Osnovni Karplus-Strong algoritam

U Karplus-Strong članku [1] detaljno je opisan osnovni algoritam. Proučimo prvo matematički zapis periodičnog tona dan koji se ponavlja svakih *p* uzoraka, dan u izrazu (1). Y_t je vrijednost t-tog uzorka.

$$Y_t = Y_{t-p} \tag{1}$$

Jednostavna modifikacija koja usrednjava dva uzastopna uzorka dana je izrazom (2). Audio signal je niz uzoraka, a uzorak može biti pozitivan i negativan realni broj (ili nula). Zato će, kao rezultat usrednjavanja, ton polagano iščeznuti.

$$Y_t = \frac{1}{2}\left(Y_{t-p} + Y_{t-p-1}\right) \tag{2}$$

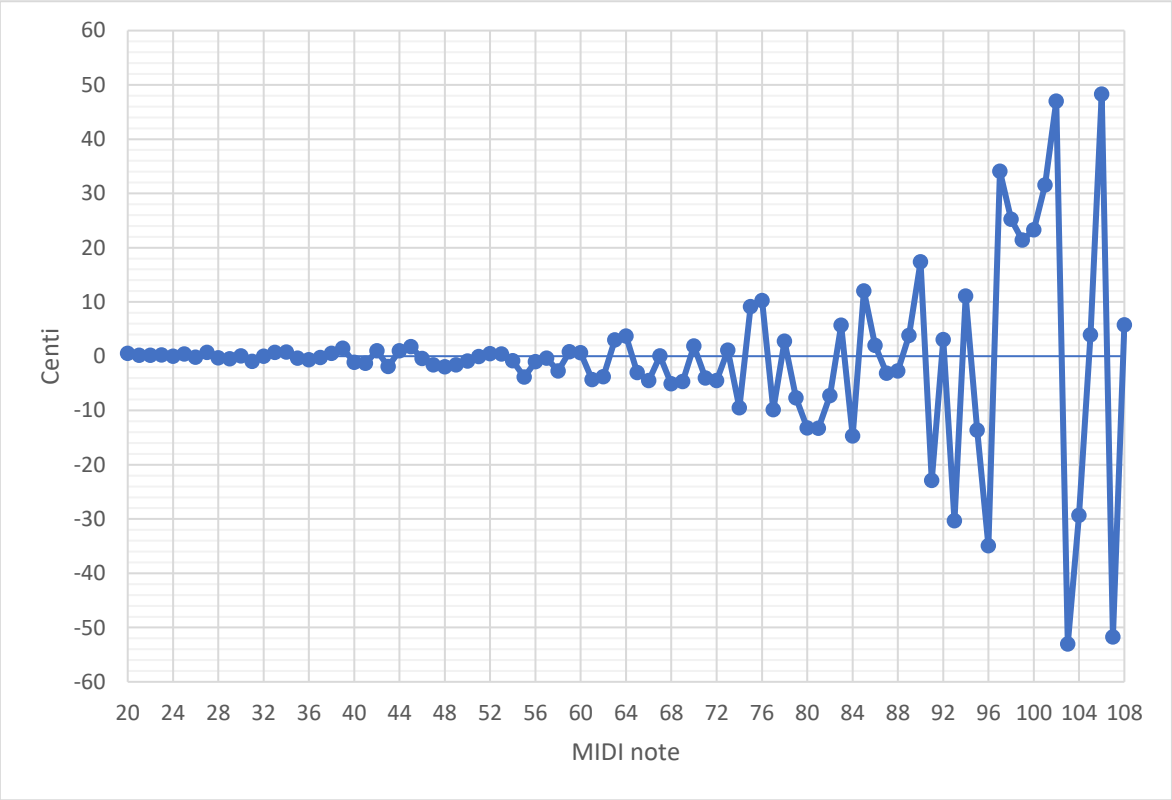
To je u principu cijeli algoritam, sve što nam treba da proizvedemo ton. Radi jako brzo na procesorima jer trebamo samo operacije zbrajanja i desnog posmaka (dijeljenje s 2). Treba nam samo međuspremnik (*buffer) u* kojeg stane p uzoraka. Frekvencija tona ovisi o frekvenciji uzorkovanja (*Sample Rate,* frekvencija reproduciranja uzoraka, najčešće 44,1 kHz) i dužini međuspremnika:

$$f_{ton} = \frac{f_{uzrokovanja}}{p + \frac{1}{2}} \tag{3}$$

Ogroman nedostatak ovakve implementacije je to što ne možemo precizno generirati željenu frekvenciju. Frekvencija uzrokovanja i duljina međuspremnika su cijeli brojevi (*integer*). Odredimo frekvenciju tona kojeg želimo sintetizirati i pomoću frekvencije uzorkovanja dobijemo potrebnu duljinu međuspremnika, koja će gotovo sigurno biti broj s decimalnim dijelom te će se morati zaokružiti na najbližu cjelobrojnu vrijednost. Zbog toga dolazi do odstupanja i „raštimanosti“, posebno kod visokih tonova. Zbog eksponencijalnog rasta

frekvencija u odnosu na tonove koje čuje ljudsko uho, razliku između dvije frekvencije nije prikladno prikazivati u hercima, nego u centima. Jedan polustepen je 100 centi. Oktavu karakterizira duplo veća frekvencija jednog tona od drugog i sastoji se od 12 polutonova. Oktava je dakle 1200 centi. Ljudi mogu čuti vrlo male razlike u tonovima, već od 5 do 6 centi [2]. Na grafu ispod (Slika 2) vidimo to odstupanje na rasponu svih tonova klavira, prikazano u centima. Razlika u centima računa se formulom danom u izrazu (4). Kao f_1 uzimamo željenu, a kao f_2 dobivenu frekvenciju.

$$\Delta centi = 1200 * \log_2 \frac{f_2}{f_1} \tag{4}$$



Slika 2 Odstupanje od željenog tona u centima

1.3. Interpolacija uzoraka

Kako bi popravili ovakva odstupanja uvodimo linearnu interpolaciju susjednih uzoraka. U osnovnoj implementaciji algoritma gledali smo amplitudu dvaju susjednih uzoraka i usrednjili ih. Možemo to gledati kao da su amplitude dvije točke između kojih povučemo crtu i na pola nje pročitamo amplitudu [3]. Međutim, mi možemo bilo gdje na toj crti pročitati novu amplitudu, npr. na pola (½), skroz lijevo (0) i skroz desno (1). Možemo to interpretirati

i na način da gledamo pomak od sredine, dodavanjem i oduzimanjem od ½. Duljinu međuspremnika p stoga možemo razdvojiti kao u izrazu (5), na cjelobrojni dio N i decimalni dio α, koji će nam reći gdje da čitamo novu vrijednost između dvaju uzoraka.

$$p = N + \alpha \tag{5}$$

Formula za linearnu interpolaciju prikazana je u izrazu (6). Primjenom supstitucije u izrazu (7), dobit ćemo željeni oblik algoritma (8).

$$Y_t = \alpha' * Y_{t-N} + (1 - \alpha') * Y_{t-N-1} \tag{6}$$

$$\alpha' = \frac{1}{2} - \frac{1}{2}\alpha \tag{7}$$

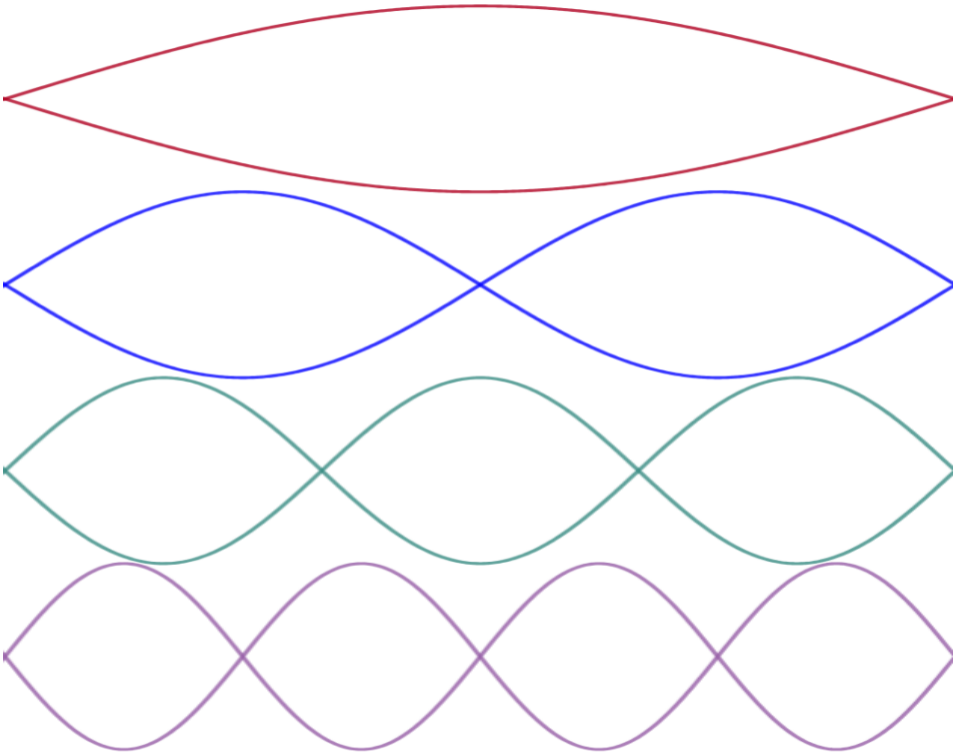
$$Y_t = \frac{1}{2}[(1 - \alpha) * Y_{t-N} + (1 + \alpha) * Y_{t-N-1}] \tag{8}$$

Pomoću ovog možemo proizvesti svaku frekvenciju, tj. svaki ton bez da bude raštiman. Bitno je napomenuti da kod ovakve implementacije, veličina međuspremnika mora biti p + 1. Ovaj oblik nam je zanimljiv, jer ako je α' = 0, dobijemo izvornu formulu (2) za osnovni algoritam.

2. Oblikovanje zvuka

2.1. Alikvotni tonovi

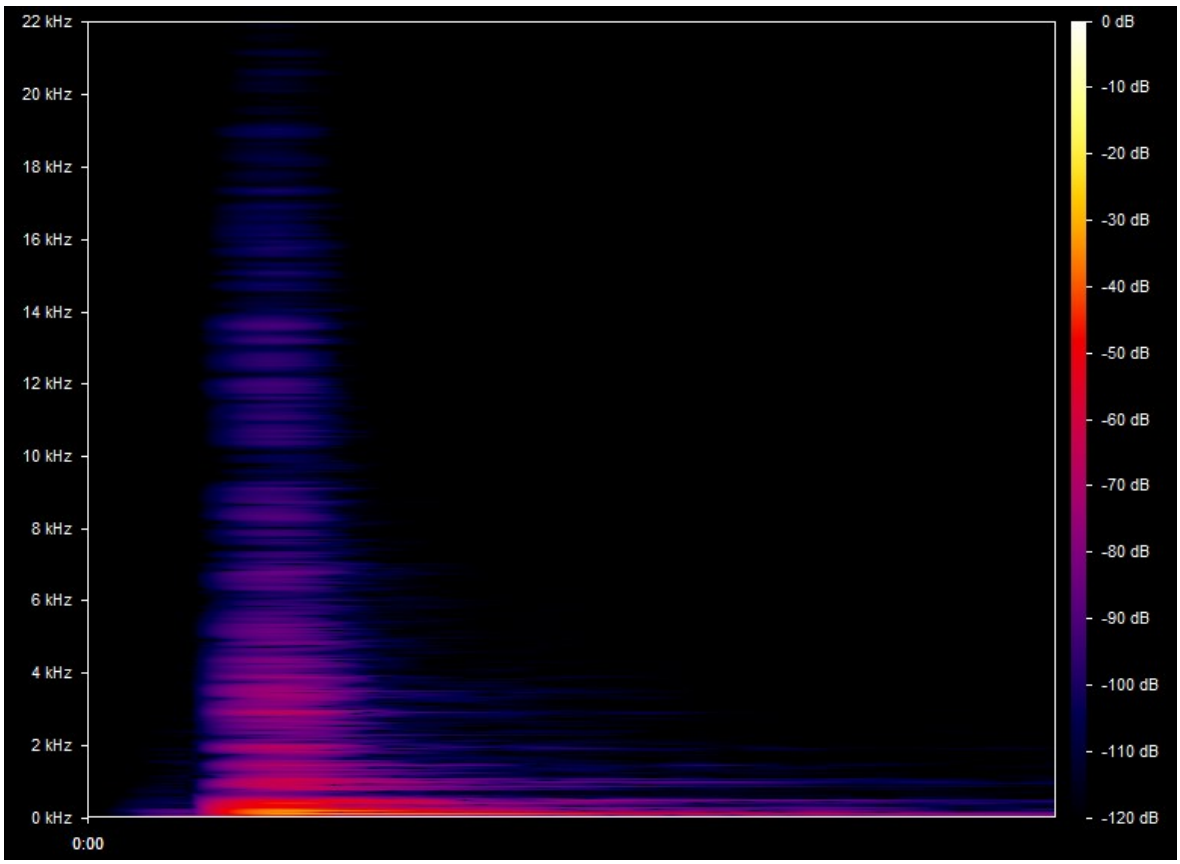
Svaki ton proizveden na fizičkom instrumentu sa sobom nosi alikvotne tonove (*alikvote*, *harmonijski niz*, engl. *aliquotes*/*overtones*) koji mu daju boju i karakter. Alikvote su tonovi koji nastaju iznad osnovnog tona. Njihovo pojavljivanje najlakše se može objasniti na prikazu žice (Slika 3), gdje osnovni ton ima brijeg u sredini i čvor na svakom od krajeva. Alikvotni tonovi također imaju čvorove na krajevima, ali i između njih. Prva alikvota ima tri čvora i dva brijega, druga ima četiri čvora i tri brijega… Ako govorimo o samoj žici, svaka alikvota ima manju amplitudu od prethodne, ali rezonantne kutije (i tijela) instrumenata imaju velik utjecaj na njihovu amplifikaciju pa to ne vrijedi uvijek, neke se ističu [4].



Slika 3 Prikaz osnovnog tona i alikvota na „žici“

Primijetimo da alikvotni niz možemo definirati kao niz tonova čija frekvencija je višekratnik frekvencije osnovnog tona. Dolazimo do zaključka da je svaki proizvedeni ton zapravo zbroj različitih tonova, tj. sinusoida. To se može i vidjeti na spektrogramu trzaja žice na tamburi

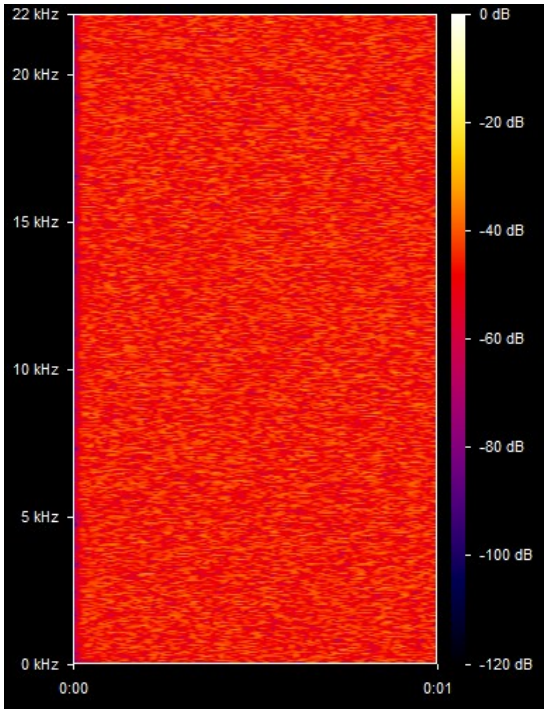
(Slika 4). Osnovni ton je najglasniji i najduži, alikvota ima puno i uglavnom je svaka tiša i kraća od prethodne.



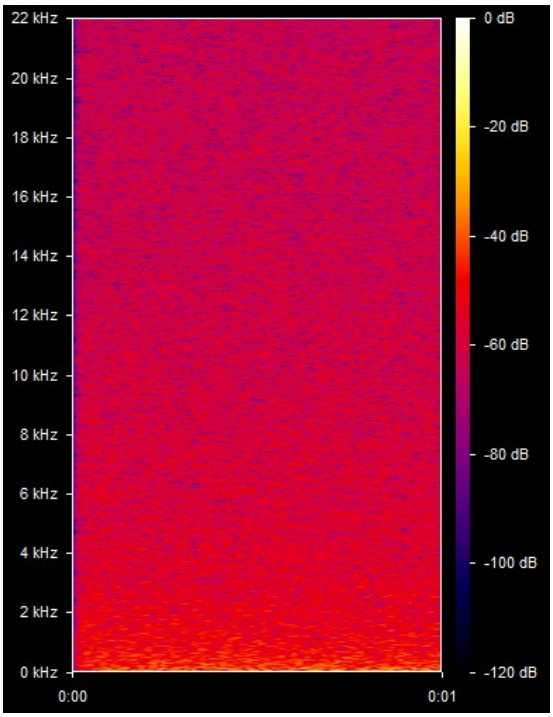
Slika 4 Spektrogram trznute žice za bugariji

2.2. Ovisnost izlaza o pobudi

Najjednostavniji impuls kojim možemo pobuditi sustav je bijeli šum (white noise), kojem je energija ravnomjerno raspoređena po svim frekvencijama (Slika 5). Kao što se spominje u 1. poglavlju, time na izlazu dobijemo agresivan, metalan ton s vrlo izraženim alikvotnim tonovima. Da bismo ublažili ton i učinili ga prirodnijim, pustit ćemo pobudu kroz nisko-propusni filtar. Ovdje možemo eksperimentirati dok ne nađemo pad filtra i graničnu frekvenciju na kojoj sinteza zvuči prirodno, bez da se ton previše zamuti. Možemo koristiti i rozi šum (pink noise, Slika 6) kojem je gustoća energije ista u frekvencijskim pojasevima s istim omjerom gornje i donje granične frekvencije. To je bliže načinu na koji čuje ljudsko uho. Sluh je logaritamski, za dva para tonova koji imaju isti omjer frekvencija kažemo da su međusobno udaljeni za isti interval [5]. Međutim, razlika u zvuku na izlazu je vrlo mala.



Slika 5 Spektrogram bijelog šuma



Slika 6 Spektrogram ružičastog šuma

Pokazalo se da se osnovni ton ili neka od njegovih alikvota neće pojaviti na izlazu algoritma ako se frekvencija iste ne nalazi u impulsu pobude. Time zaključujemo da najniža frekvencija u impulsu ne smije biti viša od najnižeg tona kojeg virtualni instrument mora moći proizvesti. Također, iako nisko-propusnim filtrom dobivamo prirodniji ton, ne smijemo primijeniti previše strmi filtar kako ne bismo previše prigušili prirodni niz alikvota sintetiziranog tona.

2.3. Vrste tambura i štimovi

Tambura je središnji instrument hrvatske tradicijske glazbe. S obzirom na to da nije svjetski korišten instrument, često nema adekvatnu reprezentaciju u svijetu virtualnih instrumenata.

Sve tambure osim bugarije se štimaju (ugađaju) kvartno te postoje dva najčešće korištena načina štimanja žica: E-A i D-G, nazvanih prema najvišoj žici instrumenata [6]. U sklopu ovog rada pričat ćemo o E-A tamburama. To znači da je najviša žica na bisernici E, na braču A, na bugariji E i na basu A. Bugarija služi za sviranje harmonijske pratnje i štima se na otvoreni E dur akord te zbog toga ima najmanji raspon.

Za bisernicu ćemo pretpostaviti da ima raspon od svoje prazne najdublje žice do zadnjeg, 20. polja na najvišoj žici. Za brač uzimamo isto, ali samo do 15. polja. Bugarija i bas do 12.

polja. Detaljan prikaz najviših i najdubljih žica te broj pragova po instrumentu vidljiv je u tablici 1.

Tablica 1 Rasponi instrumenata u MIDI vrijednostima

Instrument	Najdublja žica	Najviša žica	Broj pragova	Ton na najvišem pragu najviše žice	MIDI vrijednost najnižeg tona	MIDI vrijednost najvišeg tona
Bisernica	$c\#^1$	e^2	20	c^4	61	76+20=96
Brač	f#	a^1	15	c^3	54	69+15=84
Bugarija	e	e^1	12	e^2	52	64+12=76
Bas	$F\#_1$	A	12	a	30	45+12=57

2.4. Načini sviranja

Postoje tehnike sviranja koje moramo emulirati. Osim toga što moramo omogućiti kontrolu dinamike, najbitnije je omogućiti korisniku da bira trza li se ton ili se samo „udara“ (lupa), tj. trzne jednom i pusti da zvoni do prestanka note. Također, korisnik treba imati kontrolu nad brzinom trzanja i parametrom „ljudskosti“, odnosno slučajnog variranja u brzini i dinamici trzanja. Korisnik treba imati i kontrolu nad jačinom prigušenja povratne veze, odnosno brzinom utišavanja note kako bi dobio sintezu bližu željenoj. Uvest ćemo i jačinu prigušenja u slučaju otpuštanja tona, da i zaustavljanje, odnosno puštanje tona bude realno.

Prilikom odabira instrumenta unutar aplikacije, valja uzeti u obzir da je za bisernicu i brač zadana (*default*) vrijednost *trzanje*, a za bas i bugariju je zadana vrijednost *udaranje*, kao što je slučaj i u pravom svijetu.

2.5. Ubacivanje snimljenog impulsa tambure

Iduće poboljšanje koje možemo dodati je staviti pravi snimljeni impuls tambure kao impuls pobude. Koristimo snimljeni trzaj svake tambure. Svaki instrument daje različit frekvencijski sadržaj i vrlo pažljivo moramo odabrati kako snimamo i koji dio tona uzimamo. Hoćemo li trznuti otvorenu žicu ili ćemo ju prigušiti da čujemo samo udarac? Smijemo li uopće koristiti dio gdje se čuje udarac trzalice, jer to dodaje visoke frekvencije koje rezultiraju metalnim tonom? Znači li to da moramo trzati prstom kako bismo čuli samo

što instrument proizvede na početku sviranja? Koliko duga pobuda nam minimalno treba? Hoćemo li za snimanje koristiti mikrofon ili kontaktni mikrofon? Koliko udaljen mikrofon mora biti?

Na raspolaganju su nam snimke trzaja na instrumentima: bisernica (prim), brač (basprim), bugarija (beglajt, kontra) i bas (berda, bajs). Snimljene pobude testiramo puštanjem kroz delay efekt u Cakewalku, postupkom opisanim u poglavlju 1.1. Potvrdili smo slutnje da korištenjem samog početka trzaja, provučenog kroz blagi nisko-propusni filtar, dobivamo bolji ton nego korištenjem bijelog šuma. Sve tambure su snimane s mikrofonom što bliže tijelu, uperenim u prostor između rupe i konjića (Slika 7, informativno, u stvarnosti mikrofon puno bliže rupi). Tako dobivamo efekt blizine (*Proximity Effect*) i vrlo topao zvuk, pogodan za našu sintezu. Također nastojimo izbjegavati zvuk trzalice i zvuk udaranja žica po pragovima tako što „sviramo“ bez trzalice i ne prejako. Najbolje rezultate za brač, dobivamo trzanjem najdublje žice palcem, dok držimo sve žice lijevom rukom kako ne bi odzvanjale. Istu tehniku koristimo za pobude basa i bugarije. Žice prigušujemo jer ne želimo čuti ton koji tambura proizvodi, već kako tijelo tambure reagira na vibracije, što izlazi iz rupe.

Slika 7 Snimanje pobude na braču

Korištenjem iste tehnike snimanja, za bisernicu ne dobivamo dobre rezultate. U tom slučaju, ispostavilo se da je bolje „kucati“ po konjiću ili stražnjoj ploči instrumenta, kao na slikama 8 i 9.

Slika 8 Kucanje po konjiću bisernice

Slika 9 Kucanje po stražnjoj ploči bisernice

Tehnika sviranja bez trzalice (kao na gitari ili kontrabasu) ne koristi se u praksi zbog tvrdoće žica. Međutim, ovdje je koristimo kako bismo dobili samo odziv žice. Zvuk udaranja trzalice o žicu nije sastavni dio tona koji proizvodi instrument i rezultirao bi u izobličenim alikvotama i metalnom zvuku. U daljnjem razvoju aplikacije možemo zasebno snimiti zvuk trzalice i omogućiti korisniku podešavanje parametra koji regulira njegovu glasnoću, kako bi se dobio dodatan osjećaj realnosti.

Kako izlaz iz našeg algoritma najviše ovisi o ulaznoj pobudi, vrlo je bitno koristiti kvalitetne snimke instrumenata čije tonove sintetiziramo, jer nam to daje karakteristiku koju želimo. Možda se ovo čini kao da radimo *sampler*, no to nije istina jer koristimo vrlo kratak segment snimke, samo za pobudu sustava. Neovisno o tome koristimo li neku vrstu šuma, pilasti signal, niz nasumičnih vrijednosti ili snimku impulsa tambure, uvijek moramo ili imati pohranjenu pobudu ili je generirati, stoga nije sporno koristiti snimku.

2.6. Prigušenje povratne veze

Naš algoritam oslanja se na povratnu vezu za sintezu tonova. Osnovni algoritam ima vrlo polagano utišavanje tonova, puno duže traju nego na stvarnim instrumentima. Kako bismo dodali dozu realnosti, dodat ćemo vrlo suptilno prigušenje povratne veze. Množenje amplitude s 0.998 već daje čujan i zadovoljavajući efekt.

Promotrimo sustav ovako: svaka pobuda mora određen broj puta proći kroz algoritam. Ako proizvodimo ton niže frekvencije, njegov period je veći, a samim time duže ostaje u *petlji*.

Ovo će najviše utjecati na duboke tonove koje proizvodi bas. Kako bismo spriječili predugo odzvanjanje tonova, morat ćemo pojačati prigušenje povratne veze, uzimajući u obzir dubinu tona koji želimo proizvesti. Također, kad želimo simulirati prestanak sviranja (ton otpušten), znatno ćemo povećati prigušenje.

3. Korištene tehnologije

3.1. Razvojna okolina JUCE

JUCE (izgovara se kao juice/džus) je projekt otvorenog kôda (*open source*), najkorištenija razvojna okolina (*framework*) za razvoj audio aplikacija i proširenja (*plug-in, plugin*) [7]. Koristi se za izradu samostalnih (*standalone*) aplikacija te brojnih formata proširenja za korištenje u raznim modernim programima za glazbenu produkciju (DAW-ovima, *Digital Audio Workstation*). Vodi računa o razlikama između operacijskih sustava i formata proširenja, što znači da omogućava razvoj za više platformi istovremeno. Koristi programski jezik C++. Velika prednost je što postoji puno gotovih primjera, foruma i tutorijala, nije nepoznato područje. Za rad sa JUCE frameworkom koristi se Projucer IDE (*Integrated Developement Environment*), a za build i kompilaciju Microsoftov Visual Studio (za Windows, Xcode za macOS). *Projucer* služi za upravljanje i izvoz projekata, može generirati potrebne predloške ovisno o potrebama projekata.

3.2. Programski jezik C++

Programski jezik C++ jedan je od najpopularnijih programskih jezika. Proširenje je, a može se reći gotovo i nadskup programskog jezika C. Od njega nasljeđuje visok stupanj kontrole nad resursima i memorijom [8]. To je važno za razvoj audio proširenja jer se izvode matematičke operacije nad pojedinim uzorcima zvučnog signala. C++ nadograđuje jezik C objektno orijentiranom paradigmom i omogućava korištenje klasa i objekata.

3.3. MIDI sučelje

Musical Instrument Digital Interface je protokol koji omogućava komunikaciju između instrumenata i uređaja pomoću digitalnih poruka [9]. Prije se za MIDI komunikaciju koristio 5-pinski DIN kabel, a danas se mogu koristiti obični USB kablovi. Svaka MIDI poruka (*MIDI message*) označava jedan ton: treba li biti upaljen (*NoteOn* poruka) ili ugašen (*NoteOff* poruka) i koliko glasan treba biti (*velocity*, vrijednosti 0-127). Primijetimo da se prilikom stiskanja i otpuštanja šalju dvije poruke/događaja, ne postoji parametar „trajanje tona“. Postoji još puno poruka koje se koriste kao signali za mijenjanje zvukova i upravljanje

parametrima uređaja kojima manipuliraju (npr. MIDI klavijatura može biti spojena na modul koji samo proizvodi zvukove), no to nam nije potrebno za aplikaciju.

3.4. Audacity i Cakewalk

Za snimanje i obradu zvukova koje koristimo kao pobude, koristimo besplatne DAW programe Audacity i Cakewalk. Audacity je jednostavan i više orijentiran na signale, omogućava generiranje šumova koji se spominju u poglavlju 2.2., dok je Cakewalk moguće koristiti za ozbiljniju glazbenu produkciju, što nije potrebno u sklopu ovog rada. Oba omogućuju grafički prikaz valnog oblika, skraćivanje snimaka i njihovu tonsku obradu.

3.5. Spek

Spek je jako jednostavan i lagan alat za spektralnu analizu audio snimaka, odnosno generiranje i pregled spektrograma. Osim toga, pokazuje kodek, brzinu prijenosa (bitrate), frekvenciju uzorkovanja i broj kanala snimke, nekad točnije od kartice *Svojstva* (Properties) u Windowsu. Svi spektrogrami u ovom radu generirani su Spekom.

3.6. Sučelje VST

VST (*Virtual Studio Technology*) sučelje razvila je tvrtka Steinberg [10]. Iako nije bilo prvo sučelje koje omogućuje integraciju virtualnih efekata i instrumenata u DAW-ove, i dan danas je najkorištenije. Potaklo je razvoj virtualnih simulacija analognih efekata iz studija, simulacija instrumenata i razvoj nekih potpuno novih instrumenata. Danas koristimo najnapredniji VST3. Osim VST, često se koriste se AU, AAX i LV2 formati, a JUCE nam omogućuje da naš kôd izvezemo u svaki od njih.

3.7. ASIO driveri

ASIO (*Audio Stream Input/Output*) drivere (programska podrška) također je razvila tvrtka Steinberg [10]. ASIO je protokol za komunikaciju aplikacija i zvučnih kartica. Podržava promjenjive brzine prijenosa i frekvencije uzorkovanja. Zbog toga što omogućuju jako kratko kašnjenje sustava (nisku latenciju, *low latency*), ASIO driveri krucijalni su općenito za audio produkciju u stvarnom vremenu, ne samo za našu aplikaciju.

4. Razvoj

4.1. Program ukratko

U ovom poglavlju dano je kratko i pojednostavljeno objašnjenje rada programa. Prilikom pokretanja, main radi instancu aplikacije. Aplikacija je klasa koja sadrži parametre kojima korisnik manipulira te instance drugih klasa koje ona koristi. U našem slučaju, nakon instanciranja aplikacije, ona radi instance sintesajzera (*synthesiser, synth*) koje generiraju zvuk. Aplikacija brine o ulazima i izlazima, prima MIDI poruke i ovisno o njima govori određenim instancama sintesajzera da proizvedu ton. U ovoj implementaciji, svaka instanca sintesajzera brine o vlastitom tonu, što nam dozvoljava lako baratanje njima i veliku polifoniju (sviranje puno različitih tonova istovremeno). Kôd, odnosno opisana arhitektura, inspirirana je JUCE primjerom *AudioSynthesiserDemo.h* [7].

4.2. Klasa TamburaApp

Klasa TamburaApp je naša aplikacija. Ona omogućuje korisniku mijenjanje parametara sinteze pomoću grafičkog sučelja, čije se „crtanje“ također definira unutar ove klase. Upravlja sintesajzerima koji su opisani klasom StringSynthesiser i prilikom mijenjanja JUCE widgeta (elementi grafičkog sučelja: potenciometri, gumbi i sl.) šalje sintesajzerima nove vrijednosti koje diktiraju njihovo ponašanje. U nastavku je prikazano sve deklarirano.

```
OwnedArray<StringSynthesiser> stringSynths;
Instrument instrument = Bisernica;  //zadana (default) vrijednost
float savedSampleRate;              //spremljena frekvencija uzorkovanja

juce::Slider pickSpeedRotary;       //potenciometar za brzinu trzanja
juce::Label pickSpeedRotaryLabel;   //labela za isti

juce::Slider pickSpeedRandRotary;   //pot. za varijaciju brzine trzanja
juce::Label pickSpeedRandRotaryLabel;

juce::Slider velocityRandRotary;    //pot. za varijaciju jačine rzanja
juce::Label velocityRandRotaryLabel;

juce::Slider playDecayRotary;       //pot. za prigušenje povratne veze
```

```
juce::Label playDecayRotaryLabel;   //tijekom sviranja

juce::Slider stopDecayRotary;       //pot. za prigušenje povratne veze
juce::Label stopDecayRotaryLabel;   //prilikom prestanka sviranja

juce::ToggleButton tremoloPickingButton;  //gumb za trzanje ili lupanje

juce::AudioDeviceManager deviceManager;   //upravitelj spojenim uređajima
juce::ComboBox midiInputList;             //lista povezanih MIDI uređaja
juce::Label midiInputListLabel;
int lastInputIndex = 0;
bool isAddingFromMidiInput = false;

juce::ComboBox instrumentList;            //lista ponuđenih instrumenata
juce::Label instrumentListLabel;

juce::MidiKeyboardState keyboardState;    //stanje klavijature
juce::MidiKeyboardComponent keyboardComponent;  //klavijatura na zaslonu
```

Kôd 4.1 Deklaracija varijabli i widgeta za kontrolu parametara

Korisnik može birati MIDI ulaz i instrument koji se svira. Ako promijeni instrument, instanca aplikacije „čisti“ sve postojeće sintesajzere i radi nove (po rasponima nota u poglavlju 3.1.), govoreći im koji instrument trebaju biti da znaju koju pobudu koristiti. U tom procesu, parametri aplikacije se resetiraju na predefinirane vrijednosti odabranog instrumenta.

Ovdje trebamo nadjačati funkcije potrebne da JUCE odradi svoj dio posla: *prepareToPlay*, *getNextAudioBlock*, *releaseResources* i *resized*.

Funkcija *prepareToPlay* pohranjuje frekvenciju uzorkovanja i poziva *generateStringSynths* koja instancira sintesajzere potrebne za sviranje svih tonova na instrumentu. Pritom dohvaća vrijednosti pomičnih elemenata na grafičkom sučelju za kontrolu trzanja.

```
void prepareToPlay (int /*samplesPerBlockExpected*/, double sampleRate)
override
{
   generateStringSynths (sampleRate, pickSpeedRotary.getValue(),
      tremoloPickingButton.getToggleState(), instrument);
   savedSampleRate = sampleRate;
}
```

Kôd 4.2 Funkcija prepareToPlay

Funkciju *getNextAudioBlock* JUCE konstantno poziva, u gotovo istom obliku nalazi se u korištenom predlošku. Zvuk u računalo se uvijek oslanja na princip međuspremnika, uzorci se dohvaćaju u blokovima (tipično 64-1024). Prvo se buffer očisti od preostalog smeća i uzorcima se puni kanal po kanal. U većini slučajeva, računalo će tražiti 1 (mono) ili 2 kanala (stereo), ali funkcija uvijek puni samo i kopira vrijednosti na preostale. Za prvi kanal, proći se sve sintesajzere i pozvati *generateAndAddData* (više u 4.3) i svaki sintesajzer će dodati zvuk koji generira u *channelData*, odnosno *bufferToFill*.

```
void getNextAudioBlock (const AudioSourceChannelInfo& bufferToFill)
override
{
   bufferToFill.clearActiveBufferRegion();

   for (auto channel = 0;
        channel < bufferToFill.buffer->getNumChannels();
        ++channel)
   {
      auto* channelData = bufferToFill.buffer->getWritePointer (channel,
                          bufferToFill.startSample);

      if (channel == 0)
      {
         for (auto synth : stringSynths)
            synth->generateAndAddData (channelData,
                                       bufferToFill.numSamples);
      }
      else
      {
         memcpy (channelData,
                 bufferToFill.buffer->getReadPointer (0),
                 ((size_t) bufferToFill.numSamples) * sizeof (float));
      }
   }
}
```

Kôd 4.3 Funkcija getNextAudioBlock

Funkcija *resized* diktira izgledom grafičkog sučelja (Slika 10), odnosno rasporedom deklariranih widgeta. Dostupni parametri za podešavanje su:

- Brzina trzanja – 70-140 [ms]

- Varijacija brzine trzanja – 0-100 [%]
- Varijacija jačine trzanja – 0-100 [%]
- Decay sviranja – 0.8-1.0 , prigušenje povratne veze tijekom sviranja
- Decay zaustavljanja – 0.0-1.0, prigušenje povratne veze u trenutku prestanka sviranja
- Trzanje – označeno-neoznačeno, trzanje ili lupanje

Razdvojili smo decay sviranja i decay zaustavljanja, jer u trenutku prestanka sviranja ne smijemo samo prestati s isporukom zvuka, već se prigušenje mora znatno povećati kako bi ton stao na realan način.

Slika 10 Grafičko sučelje aplikacije

4.3. Klasa StringSynthesiser

Kod instanciranja klase StringSynthesiser, konstruktor prima parametre:

- double sampleRate = frekvencija uzorkovanja računala (npr. 44100)
- double frequencyInHz = frekvencija tona kojeg generira (npr. 440)

- int pickSpeed = brzina trzanja (70-140)
- bool trzanje = trzanje (true) ili lupanje (false)
- Instrument instrument = instrument kojeg generira

Globalno smo uveli enum *Instrument* koji poprima vrijednosti *Bisernica*, *Brac*, *Bugarija* i *Bas*. Svaki parametar ima setter (*set* funkciju) kojim ga TamburaApp može mijenjati (*settati*, postaviti). Prilikom instanciranja klase StringSynthesiser, poziva se funkcija *prepareSynthesiserState* koja prvo spremi bitne parametre (frekvenciju uzorkovanja, frekvenciju generiranog tona i instrument) te računa cijeli i decimalni dio veličine međuspremnika kao u izrazu (5). Nakon toga, poziva funkciju *loadInstrumentToVector* koja puni međuspremnik *excitationSample* potrebnom pobudom ovisno o instrumentu (enum) kojeg primi.

```
void prepareSynthesiserState (double sampleRate, double frequencyInHz,
Instrument instrument)
{
   savedSampleRate = sampleRate;
   this->frequencyInHz = frequencyInHz;
   this->instrument = instrument;
   delayLineWhole = (int) std::floor(sampleRate / frequencyInHz);
   delayLineDecimal = (float) (sampleRate / frequencyInHz)
                      - (float) delayLineWhole;

   delayLine.resize(delayLineWhole + 1);
   std::fill (delayLine.begin(), delayLine.end(), 0.0f);
   excitationSample.resize(delayLineWhole + 1);
   loadInstrumentToVector(instrument);
}
```

Kôd 4.4 Funkcija prepareSynthesiserState

Kad TamburaApp primi *NoteOn* poruku, poziva funkciju *stringPlucked*. U uvjetu if-a provjeravamo istinitost varijable *doPluckForNextBuffer*. Ako je 1, postavljamo ga u 0 (važno zbog funkcije *generateAndAddData*) i izvršavamo tijelo if-a. Unutra postavljamo vrijednost amplitude ovisno o jačini pritiska tipke (*velocity*) i dodajemo ljudski faktor *velocityRand*. Parametar *decay* koristimo za manipulaciju sviranja, odnosno prestanka sviranja tona te ga postavljamo na vrijednost korisnički postavljenog *playDecay* ili *stopDecay*. Ako je trzanje true, pokreće se timer koji će ponovno pokrenuti funkciju ovisno

o *pickSpeed* kojem isto dodajemo ljudski faktor *pickSpeedRand*. Prilikom prestanka sviranja tona (*NoteOff* poruka), timer se zaustavlja, a *decay* vraća na *stopDecay*.

```
void stringPlucked()
{
   if (doPluckForNextBuffer.compareAndSetBool (1, 0))
   {
      amplitude = std::sin(MathConstants<float>::pi * 0.5 * velocity) *
         (0.5 + juce::Random::getSystemRandom().nextDouble() *
         ((float)velocityRand * 0.01));
      decay = playDecay;
      if (trzanje == true)
         startTimer(pickSpeed + (float)pickSpeedRand *
         juce::Random::getSystemRandom().nextDouble());
   }
}

void timerCallback()
{
   if (trzanje)    //ako se promijeni usred sviranja
      stringPlucked();
}

void stringMuted()
{
   stopTimer();
   decay = stopDecay;
}
```

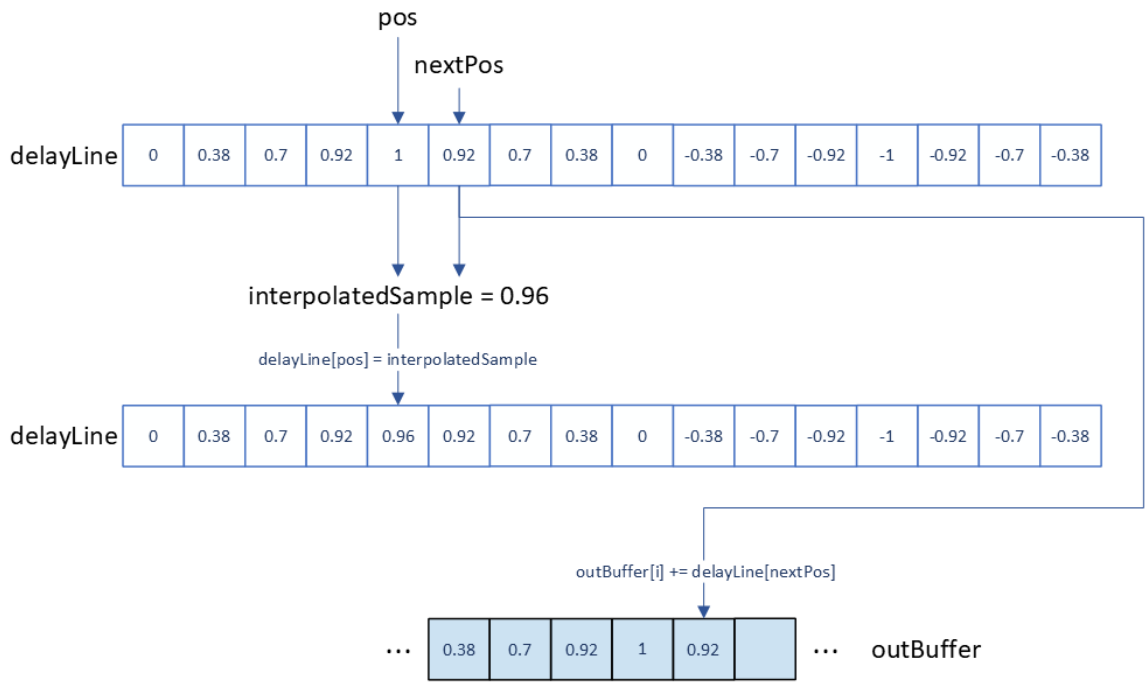
Kôd 4.5 Funkcije za početak, nastavak i završetak sviranja tona

Osim funkcije *stringPlucked*, najbitnija funkcija koja se cijelo vrijeme izvršava prilikom dohvaćanja audio blokova za izlaz iz programa je *generateAndAddData*. Ovdje pozivamo funkciju *exciteInternalBuffer* ako je *doPluckForNextBuffer* 0, ako smo ga postavili u funkciji *stringPlucked*. Neovisno o tome, u for petlji se generira niz uzoraka za idući audio blok koji dohvaćamo. Broj uzoraka je *numSamples*, veličina audio međuspremnika s kojom računalo trenutačno radi, odnosno broj uzroraka koji se obradi prije nego što se šalje na izlaz. Što je taj broj manji, to je zahtjevniji rad procesora jer mora sve obaviti u vrlo kratkom roku. Unutar same for petlje vidimo da interpolatedSample poprima vrijednost idućeg uzorka prema interpolaciji koju smo spominjali u poglavlju 1.3, danoj u izrazu (5). Ovdje vidimo dva

međuspremnika: *delayLine* je kružni spremnik čija duljina ovisi o frekvenciji tona i uzorkovanja (opisivan u poglavlju 1.2 i 1.3), a *outBuffer* je pokazivač na izlaz u kojeg „ispisujemo“ uzorke. Grafički prikaz punjenja međuspremnika nalazi se ispod kôda (Slika 11). Indeks *pos* se cijelo vrijeme pomiče naprijed kroz *delayLine* i sačuvan je kao varijabla StringSynthesisera. Funkcija vodi računa o prelasku s kraja na početak buffera. Modifikacija (interpolacija) koju napravimo u *delayLine* bufferu dolazi na izlaz *outBuffer* tek u idućem krugu obilaska *delayLine*-a. U *outBuffer* se dodaje vrijednost, a ne zamjenjuje, jer je taj međuspremnik dijeljen među svim sintesajzerima.

```
void generateAndAddData (float* outBuffer, int numSamples)
{
   if (doPluckForNextBuffer.compareAndSetBool (0, 1))
       exciteInternalBuffer();
   for (auto i = 0; i < numSamples; ++i)
   {
      auto nextPos = (pos + 1) % delayLine.size();
      float interpolatedSample =
         decay * 0.5f *
         ((1.0f - delayLineDecimal) * delayLine[nextPos] +
          (1.0f + delayLineDecimal) * delayLine[pos]);
      delayLine[pos] = interpolatedSample;
      outBuffer[i] += delayLine[nextPos];
      pos = nextPos;
   }
}
```

Kôd 4.6 Funkcija generateAndAddData



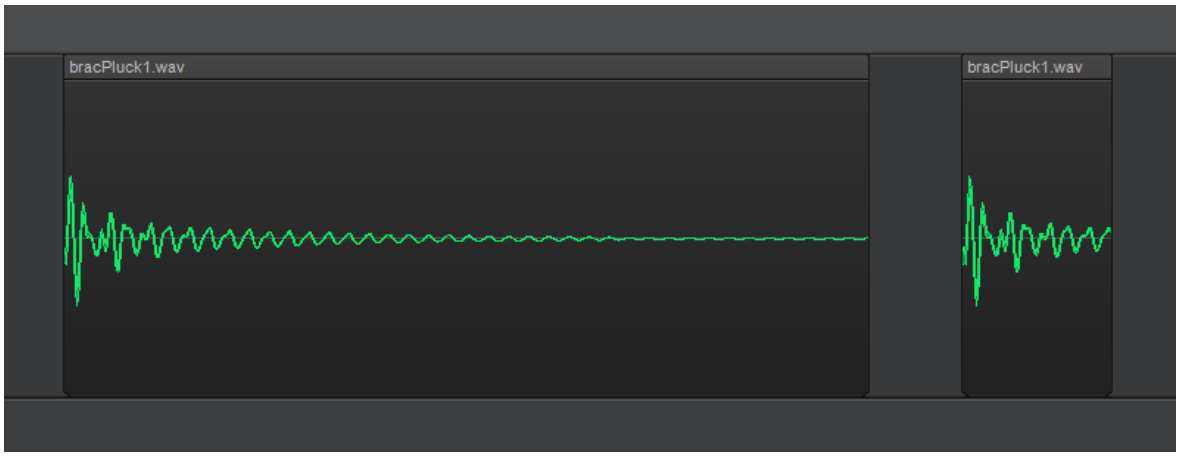
Slika 11 Interpolacija i punjenje međuspremnika

Iduća bitna funkcija je *exciteInternalBuffer*, koju poziva *generateAndAddData* ako treba krenuti sa sintezom tona (ton je stisnut/trznut). Na početku provjeravamo je li ton dovoljno dugačak da popuni cijeli buffer (TODO: nije gotova funkcija, assert zaustavlja izvođenje). Ako je, popunjavamo prazan *delayLine* (kružni spremnik) pobudom i tako kreće sinteza, aktivacija sustava. Ako je *excitationSample* duži od *delayLine*, krati se, a ako je kraći, ostatak se puni nulama.

```
void exciteInternalBuffer()
{
   jassert (delayLine.size() >= excitationSample.size());
   for (size_t i = 0; i < delayLine.size(); ++i)
   {
      delayLine[i % delayLine.size()] =
         static_cast<float>(amplitude * excitationSample[i] *
         ((delayLine.size() - i) < (delayLine.size() * 0.25f)
            ? (delayLine.size() - i) / (delayLine.size() * 0.25f)
            : 1)
         * (i < 10 ? i / 10.0f : 1));
   }
}
```

Kôd 4.7 Funkcija exciteInternalBuffer

Pri punjenju međuspremnika *delayLine* dolazimo do problema, jer ako funkcija odreže *excitationSample* u trenutku dok mu vrijednost nije oko nule (Slika 12, desno), opet dobijemo metalni ton s puno alikvota, stoga dodajemo lagani „fade-out“ na zadnjih 25% uzoraka. U slučaju punjenja nulama nemamo problema, jer se jasno vidi da vrijednosti uzoraka snimljene pobude već teže u nulu.



Slika 12 Kraćenje pobude prilikom punjenja međuspremnika

U nastavku vidimo primjer potrebne duljine *excitationSample*-a u broju uzoraka, u slučaju kad je frekvencija uzorkovanja računala 44.1 kHz (Tablica 2). Prisjetimo se poglavlja 1, duljina međuspremnika *delayLine* računa se izrazom (7). Veličina međuspremnika za najviši ton na bisernici je 22, što je vrlo malo, pogotovo kad pričamo o audio signalu. Pitanje do koje točke će funkcija obuhvatiti snimljenu pobudu, jer ako ne dođe ni do prvog vrha u audio signalu, izlaz će biti sve tiši za više tonove. Stupac „Veličina međuspremnika *delayLine* za najniži ton“ daje nam dobru ideju o koliko najmanje uzoraka mora biti dugačka snimljena pobuda koju koristimo za sintezu. Naravno, moramo računati na to da neki sustavi koriste veće frekvencije uzorkovanja, no to ćemo pokriti korištenjem znatno dužih snimki te dodavanjem nula u slučaju da uzorak i dalje nije dovoljno dugačak.

$$delayLine.size() = \left\lfloor \frac{f_{uzorkovanja}}{f_{ton}} \right\rfloor + 1 \tag{7}$$

Tablica 2 Veličine međuspremnika pri 44.1 kHz ovisno o visini tona

Instrument	MIDI vrijednost najnižeg tona	Frekvencija najnižeg tona	Veličina međuspremnika *delayLine* za najniži ton	MIDI vrijednost najvišeg tona	Frekvencija najvišeg tona	Veličina međuspremnika *delayLine* za najviši ton

Bisernica	61	277.183	160	96	2093.005	22
Brač	54	185.997	239	84	1046.502	43
Bugarija	52	164.814	268	76	659.255	67
Bas	30	46.249	954	57	220	201

Zaključak

Karplus-Strong je algoritam široke primjene i daje zadovoljavajuće rezultate kod sinteze zvuka tambure. Korištenjem pobuda snimljenih na pravim tamburama i podešavanjem faktora prigušenja povratne veze može se dobiti realan zvuk tambure. Prigušenje povratne veze za niže tonove mora biti jače jer im period ima duže trajanje pa duže ostaju u petlji. Glavnu manu osnovnog Karplus-Strong algoritma, to da generira samo približne frekvencije zbog cjelobrojne veličine međuspremnika, riješili smo uvođenjem linearne interpolacije uzoraka. Dodali smo parametre za varijaciju brzine i jačine trzanja, čijim podešavanjem dobivamo dodatnu realnost i element ljudskosti. U budućnosti ima mjesta za pretvaranje aplikacije u VST dodatak korištenjem drugog, kompliciranijeg predloška za projekt. Također se može uvesti modeliranje pobude i rezonantne kutije te suptilne razlike u zvuku ovisno o smjeru trzanja. Štoviše, mogao bi se primijeniti algoritam strojnog učenja koji na temelju snimljenih zvukova tonova na tamburama uči kako napraviti najbolju pobudu. Potreba za ovakvim programima postoji, ali zbog malog tržišta, nema puno inicijative. Ovim radom smo jedan korak bliže adekvatnoj reprezentaciji tambure u svijetu virtualnih instrumenata.

Literatura

[1] Karplus, K., Strong, A., *Digital Synthesis of Plucked-String and Drum Timbres*. Computer Music Journal, 7,2 (1983), str. 43-55.

[2] Loeffler, D.B., *Instrument Timbres and Pitch Estimation in Polyphonic Music*. Diplomski rad. Georgia Institute of Technology School of Electrical and Computer Engineering, 2006.

[3] Smith, J.O., *Physical Audio Signal Processing*. Online knjiga. 2010. Poveznica: http://ccrma.stanford.edu/~jos/pasp/, pristupljeno 18. 6. 2025.

[4] Curtu, I., Stanciu, M.D., *Experimental research regarding the dynamic behaviour of the classical guitar*. Department of Strength of Materials and Mechanical Vibrations Transilvania University of Braşov, 2009.

[5] Jelaković, T., *Zvuk, sluh, arhitektonska akustika*. Školska knjiga, Zagreb, 1978.

[6] Ferić, M., *Hrvatski tamburaški brevijar*. Šokadija, Zagreb, 2011.

[7] JUCE, GitHub repozitorij. Poveznica: https://github.com/juce-framework/JUCE, pristupljeno 18. 6. 2025.

[8] W3Schools, C++ Introduction. Poveznica: https://www.w3schools.com/cpp/cpp_intro.asp, pristupljeno 18. 6. 2025.

[9] Avid, Ultimate guide to using MIDI in music production. https://www.avid.com/resource-center/what-is-midi-musical-instrument-digital-interface, pristupljeno 18. 6. 2025.

[10] Steinberg, Our Technologies. Poveznica: https://www.steinberg.net/technology/, pristupljeno 8. 6. 2025.

Sažetak

Razvoj VST dodatka za sintezu zvuka tambure

Rad se bavi primjenom Karplus-Strong algoritma na sintezu zvukova tambure. Raspravlja o problematici cjelobrojne veličine međuspremnika i kako to utječe na preciznost generiranja tonova. Pomoću razvojne okoline JUCE, implementira se dorađena verzija Karplus-Strong algoritma i dodaje kontrola parametara koji diktiraju ponašanje sintesajzera. Dodatna realnost zvuka postiže se korištenjem snimljenih pobuda s pravih instrumenata te parametrima za varijaciju brzine i jačine trzanja.

Tambura, sinteza zvuka, Karplus-Strong, JUCE, VST

Summary

Development of a VST plug-in for tambourine sound synthesis

This thesis examines the application of the Karplus-Strong algorithm to synthesise sounds of the tambura. It discusses the issue of buffer size being an integer and how it affects the frequency of the generated note. Using the JUCE framework, a refined version of Karplus-Strong is implemented, with added support for parameter control to dictate the behaviour of synthesisers. An added element of realism is achieved by using recorded excitation samples from real instruments and enabling control of variation in picking speed and strength.

Tambura, sound synthesis, Karplus-Strong, JUCE, VST